Contenido

Introducción 3
Una biblioteca de pruebas 3
Un framework de pruebas 3
MSTest.sdk 3
Objetivos del manual 3
Ecosistema de pruebas de Microsoft (recomendado) 4
MSTest.sdk 5
¿Qué es un SDK? 5
Diferencia entre SDK y Framework 5
Ventajas de utilizar MSTest.Sdk 5
Ejemplo del fichero **.csproj** para un proyecto de pruebas basado en MSTest.sdk 5
Crear un proyecto de pruebas con Visual Studio 2026 6
Convenciones sobre el nombre 6
Creación de un proyecto 6
Nombre del proyecto de pruebas 6
Proceso de creación de forma gráfica 6
Proceso de creación en modo comando desde CLI 6
MSTest Framework 7
Creación de una clase de pruebas en MSTest 7
Atributos de Identificación 7
Atributos de ciclo de vida: inicialización y limpieza 8
Buenas prácticas 8
Atributos de parametrización 8
Atributo [DataRow] 8
Atributo [DynamicData] 9
Buenas prácticas 9
Ventajas 9
Atributos de clasificación y metadatos 9
Atributo [TestCategory] 10
Atributo [TestProperty("Atributo", "Valor")] 10
Atributos de control de ejecución 11

Atributo Ignore 11
Atributo [Timeout] 11
Excepciones 12
Atributo [ExpectedException(typeof(Exception))] 12
Limitaciones 12
Por este motivo actualmente suele preferirse la “Aserción”: 12
Principales aserciones de MSTest 12
Assert.AreEqual 12
Assert.AreNotEqual 13
Assert.IsTrue 13
Otras aserciones 13
Aserciones para cadenas 14
Aserciones para colecciones 14
Enlaces de interés sobre aserciones 14
Microsoft Code Coverage 14
Ejecutar las pruebas con análisis de cobertura 15
Desde Visual Studio 15
Desde la línea de comandos: 15
Analizar los resultados 15
Localizar métodos que no han sido ejecutados durante las pruebas 15
Mejorar la cobertura 16
Interpretación de los resultados 16
Excluir código del análisis 16
Cómo se visualiza la cobertura 16
Representación de la cobertura 17
Explorador de pruebas 17
Apertura del Explorador de pruebas 17
Principales funcionalidades 17
Ejecutar pruebas 17
Depurar pruebas 18
Filtrar pruebas 18
Consultar resultados 18

Introducción

Las pruebas unitarias constituyen uno de los pilares del desarrollo moderno de software. Permiten verificar de forma automática que una unidad de código, normalmente un método o una clase, se comporta según lo esperado ante distintos escenarios.

Para implementar estas pruebas existen herramientas especializadas que suelen agruparse en dos categorías: bibliotecas y frameworks de pruebas.

Una biblioteca de pruebas

Una biblioteca proporciona funcionalidades reutilizables que el desarrollador invoca directamente desde su código. Por ejemplo, una biblioteca puede ofrecer métodos para realizar comparaciones, generar datos de prueba o simular comportamientos.

Un framework de pruebas

Un framework, en cambio, proporciona una infraestructura completa para la definición, organización, ejecución y análisis de las pruebas. La principal diferencia es que el framework controla el proceso de ejecución y es quien invoca el código de prueba cuando corresponde.

Esta característica se conoce como inversión de control. Mientras que en una biblioteca la aplicación llama a la funcionalidad que necesita, en un framework es el propio framework quien descubre y ejecuta las pruebas definidas por el desarrollador.

MSTest.sdk

Lo primero, es importante indicar que **MSTest.Sdk** no es el framework de pruebas.

MSTest.Sdk es el mecanismo que configura el proyecto para que framework "**M**STes**t**" pueda utilizarse de forma sencilla y coherente.

En el ecosistema .NET, "**MSTest**" es el framework de pruebas unitarias mantenido por Microsoft, proporciona los atributos, las aserciones y el ciclo de vida necesarios para crear pruebas automatizadas, así como la integración con Visual Studio, la CLI de .NET y los sistemas de integración continua.

Objetivos del manual

- Conocer la función de MSTest.Sdk dentro del ecosistema moderno de pruebas de Microsoft.
- Crear y configurar un proyecto básico de pruebas unitarias con MSTest.Sdk.
- Identificar la estructura habitual de una clase de pruebas y los principales atributos utilizados por MSTest.
- Diseñar y ejecutar pruebas unitarias sencillas utilizando aserciones básicas.
- Aplicar las categorías de pruebas para organizar y filtrar la ejecución de los tests.
- Crear pruebas dirigidas por datos mediante los mecanismos proporcionados por MSTest.
- Verificar el lanzamiento de excepciones y validar escenarios de error.
- Ejecutar y depurar pruebas desde Visual Studio.
- Interpretar los resultados obtenidos tras la ejecución de una suite de pruebas.

- Comprender el concepto de cobertura de código y analizar métricas básicas de cobertura para identificar partes del código que no han sido ejercitadas por las pruebas.

Ecosistema de pruebas de Microsoft (recomendado)

Componentes vinculados con las pruebas y su función:

Componente	*Función*
MSTest	MSTest es el framework de pruebas unitarias de Microsoft para .NET. Su función es proporcionar los elementos necesarios para definir pruebas, como atributos ([TestClass], [TestMethod], [DataRow]) y aserciones (Assert.AreEqual, Assert.IsTrue, etc.). Cuando un desarrollador escribe una prueba, realmente está utilizando las clases y atributos que proporciona MSTest.
Microsoft Testing Platform	Microsoft Testing Platform es la infraestructura encargada de descubrir, ejecutar y gestionar las pruebas. Cuando se ejecuta el comando dotnet test o se solicita la ejecución desde Visual Studio, esta plataforma localiza las pruebas definidas por MSTest (u otros frameworks compatibles), las ejecuta y genera los resultados correspondientes. Puede considerarse el motor de ejecución del ecosistema moderno de pruebas de Microsoft.
Microsoft Code Coverage	Microsoft Code Coverage es una extensión de Microsoft Testing Platform que mide qué partes del código fuente han sido ejecutadas durante las pruebas. No crea pruebas ni las ejecuta; simplemente observa la ejecución realizada por la plataforma y genera métricas e informes de cobertura. Gracias a ello es posible detectar métodos, ramas o líneas de código que todavía no están siendo verificadas por las pruebas automatizadas.
Explorador de pruebas	El Explorador de pruebas es la herramienta gráfica incluida en Visual Studio para trabajar con las pruebas. Permite visualizar las pruebas detectadas, ejecutarlas, depurarlas, filtrarlas por categorías y consultar sus resultados. No ejecuta directamente las pruebas: actúa como interfaz de usuario sobre Microsoft Testing Platform, mostrando de forma cómoda la información generada durante la ejecución.
MSTest.Sdk	MSTest.Sdk es el SDK que simplifica la configuración de un proyecto de pruebas moderno. Su misión consiste en conectar y configurar automáticamente los distintos componentes necesarios, incluyendo MSTest, Microsoft Testing Platform y otras extensiones compatibles como Code Coverage. Gracias a MSTest.Sdk el desarrollador no necesita configurar manualmente todos estos componentes en el archivo .csproj.

MSTest.sdk

¿Qué es un SDK?

SDK son las siglas de **Software Development Kit** o Kit de Desarrollo de Software.

Un SDK es un conjunto de herramientas, configuraciones, bibliotecas y utilidades que facilitan el desarrollo de aplicaciones o componentes para una tecnología concreta.

Su objetivo es que el desarrollador no tenga que configurar manualmente todos los elementos necesarios para empezar a trabajar. Proporciona:

- Configuración predeterminada.
- Dependencias necesarias.
- Herramientas de compilación.
- Herramientas de ejecución.
- Plantillas y convenciones recomendadas.

Diferencia entre SDK y Framework

Concepto	Función
Biblioteca	Proporciona funcionalidades reutilizables.
Framework	Proporciona una estructura de trabajo y controla la ejecución.
SDK	Proporciona la configuración y herramientas necesarias para desarrollar utilizando un framework o una plataforma.

Ventajas de utilizar MSTest.Sdk

- Simplifica la creación de proyectos de pruebas.
- Reduce la configuración manual.
- Facilita el mantenimiento de dependencias compatibles.
- Favorece una configuración homogénea entre proyectos.
- Se adapta al ecosistema moderno de pruebas de Microsoft.

Ejemplo del fichero .csproj para un proyecto de pruebas basado en MSTest.sdk

```
<Project Sdk="MSTest.Sdk/3.10.2">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Carpeta\Proyecto.csproj" />
```

```
  </ItemGroup>

</Project>
```

Crear un proyecto de pruebas con Visual Studio 2026

Al crear un proyecto de pruebas mediante la plantilla Proyecto de pruebas MSTest de Visual Studio, el componente principal que se está utilizando es MSTest.Sdk, ya que es el encargado de configurar el proyecto y de integrar opcionalmente componentes como Microsoft Testing Platform para la ejecución de pruebas y Microsoft Code Coverage para el análisis de cobertura.

Visual Studio actúa únicamente como la interfaz gráfica que simplifica este proceso de configuración.

Convenciones sobre el nombre

MSTest descubre las pruebas por los atributos ([TestClass], [TestMethod]), no por el nombre del proyecto. Sin embargo, la convención más utilizada en .NET es: **NombreProyecto.Tests**

Creación de un proyecto

Requisitos

Para la creación del proyecto de pruebas del ejemplo, se presupone que:

1. Existe una solución denominada "**TestProjectMangementPlatform**" que está localizada en una carpeta con el mismo nombre dentro de la carpeta de repositorio predeterminada.
2. Disponemos de un proyecto de tipo "**Biblioteca de clases**" denominada "**Nucleo**" (Core en inglés) localizada en una carpeta con el mismo nombre localizada en la carpeta de la solución.

Nombre del proyecto de pruebas

MSTest descubre las pruebas por los atributos ([TestClass], [TestMethod]), no por el nombre del proyecto. Sin embargo, la convención más utilizada en .NET es: **NombreProyecto.Tests**

Proceso de creación de forma gráfica

1. Hacer clic derecho sobre la solución y seleccionar: **Agregar → Nuevo proyecto**
2. Buscar y seleccionar la plantilla: **Proyecto de pruebas MSTest**
3. Asignar un nombre al proyecto: **Nucleo.Tests**
4. Seleccionar la opción "**usar MSTest.sdk**" y otras opciones como **MicrosoftTestingPlatform** para la plataforma de ejecución y **MicrosoftCodeCoverage** para la de cobertura y pulsar **Crear**.
5. Posteriormente debemos añadir una referencia al proyecto que se desea probar mediante: **Agregar → Referencia de proyecto** en el menú contextual del apartado **Dependencias** dentro del proyecto de pruebas.

Proceso de creación en modo comando desde CLI[1]

Se incluyen a modo ilustrativo comandos adicionales para compilar y ejecutar las pruebas con y sin la generación de cobertura.

[1] CLI son las siglas de Command-Line Interface (Interfaz de Línea de Comandos).

```
# Situarse en la carpeta de la solución
cd %USERPROFILE%\source\repos\TestProjectMangementPlatform

# Crear el proyecto de pruebas
dotnet new mstest -n Nucleo.Tests -o Nucleo.Tests

# Añadir el proyecto de pruebas a la solución
dotnet sln TestProjectMangementPlatform.sln add Nucleo.Tests\Nucleo.Tests.csproj

# Añadir una referencia al proyecto que se desea probar
dotnet add Nucleo.Tests\Nucleo.Tests.csproj reference Nucleo\Nucleo.csproj

# Instalar Microsoft Code Coverage
dotnet add Nucleo.Tests\Nucleo.Tests.csproj package
Microsoft.Testing.Extensions.CodeCoverage

# Compilar la solución
dotnet build

# Ejecutar las pruebas
dotnet test

# Ejecutar las pruebas generando cobertura
dotnet test --coverage
``
```

MSTest Framework

Creación de una clase de pruebas en MSTest

Las pruebas en MSTest se organizan en clases de prueba. Cada clase contiene uno o varios métodos encargados de verificar distintos comportamientos de la aplicación.

La estructura mínima es la siguiente:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace Nucleo.Tests;

[TestClass]
public class CalculadoraTests
{
    [TestMethod]
    public void Sumar_DosNumeros_DevuelveResultadoCorrecto()
    {
        Assert.AreEqual(5, 2 + 3);
    }
}
```

Atributos de Identificación

[TestClass]	identifica una clase que contiene pruebas.
[TestMethod]	*identifica un método que representa una prueba individual.*

Atributos de ciclo de vida: inicialización y limpieza

En una suite de pruebas es habitual que varias pruebas necesiten preparar objetos, configurar datos o liberar recursos. Para evitar duplicar este código en cada prueba, MSTest proporciona atributos especiales que permiten ejecutar métodos automáticamente antes y después de las pruebas.

En la tabla siguiente se muestran los atributos, la frecuencia y su uso recomendado

Atributo	Frecuencia	Uso recomendado
[TestInitialize]	Antes de cada prueba	Crear objetos limpios para cada prueba
[TestCleanup]	Después de cada prueba	Liberar recursos específicos de una prueba
[ClassInitialize]	Una vez por clase antes de la primera prueba	Preparaciones costosas que pueden compartirse
[ClassCleanup]	Una vez por clase después de la última prueba	Liberar recursos compartidos

Buenas prácticas

Mantener las pruebas independientes

Cada prueba debe poder ejecutarse individualmente y obtener siempre el mismo resultado. Por este motivo, normalmente es preferible usar: ***[TestInitialize]*** en lugar de compartir estado entre pruebas.

Evitar dependencias entre pruebas

Nunca debe asumirse que una prueba se ejecutará antes que otra.

Usar [ClassInitialize] solo cuando aporte un beneficio real

La compartición de recursos mejora el rendimiento, pero también aumenta el riesgo de que una prueba afecte a otra. Por ello, para pruebas unitarias simples suele bastar con : ***[TestInitialize]*** y ***[TestCleanup]***.

Atributos de parametrización

Los atributos de parametrización permiten ejecutar una misma prueba varias veces utilizando distintos conjuntos de datos: ***[DataRow]*** y ***[DynamicData].***

Su principal objetivo es evitar la duplicación de código y facilitar el mantenimiento de las pruebas. El siguiente código incluye un ejemplo.

Atributo [DataRow]

```
[TestMethod]
[DataRow(1, 2, 3)]
[DataRow(2, 3, 5)]
[DataRow(-1, 1, 0)]

public void Sumar_Valores (
    int a,
```

```
    int b,
    int esperado)
{
    Assert.AreEqual(esperado, a + b);
}
```

Atributo [DynamicData]

```
public static IEnumerable<object[]> CasosPrueba()
{
    yield return new object[] { 1, 2, 3 };
    yield return new object[] { 2, 3, 5 };
    yield return new object[] { 10, 5, 15 };
}

[TestMethod]
[DynamicData(nameof(CasosPrueba))]
public void Sumar_Valores (
    int a,
    int b,
    int esperado)
{
    Assert.AreEqual(esperado, a + b);
}
```

Buenas prácticas

Se recomienda usar ***[DataRow]*** cuando

- Los datos son pocos.
- Los datos son sencillos.
- Los valores pueden escribirse directamente en el código.

Se recomienda usar ***[DynamicData]*** cuando

- Gran cantidad de casos de prueba.
- Datos generados dinámicamente.
- Reutilización de conjuntos de datos entre varias pruebas.

Ventajas

- Reduce la duplicación de pruebas.
- Facilita añadir nuevos casos.
- Permite identificar rápidamente qué conjunto de datos provoca un error.

Atributos de clasificación y metadatos

En proyectos pequeños, ejecutar todas las pruebas suele ser viable. Sin embargo, a medida que una aplicación crece, pueden existir cientos o miles de pruebas. En estas situaciones resulta imprescindible disponer.

MSTest proporciona varios atributos destinados a la clasificación y documentación de las pruebas: ***[TestCategory]*** y ***[TestProperty]***.

Atributo [TestCategory]

Las categorías permiten:

- Organizar la suite de pruebas.
- Filtrar las pruebas durante la ejecución.
- Ejecutar subconjuntos específicos.
- Diferenciar distintos niveles de prueba.
- Integrar pruebas en pipelines CI/CD.

Una misma prueba puede pertenecer a una o simultáneamente a varias categorías.

```
[TestMethod]
[TestCategory("Unitarias")]
[TestCategory("Usuarios")]
[TestCategory("Seguridad")]
[TestCategory("Regresión")]
public void ValidarPassword_PasswordValida_Aceptada()
{
}
```

Ejemplos:

- Por nivel de pruebas: Unitarias, Integración, Sistema, Aceptación, Regresión
- Por módulo funcional: Usuarios, Proyectos, Autenticación, Seguridad
- Por tiempo de ejecución: Rápidas, Lentas
- Por dependencias externas: BaseDatos, API, ServiciosExternos

Atributo [TestProperty("Atributo", "Valor")]

A diferencia de ***[TestCategory]***, que está orientado a la clasificación, ***[TestProperty]*** sirve para documentar información adicional.

```
[TestMethod]
[TestProperty("Autor", "Pedro")]
[TestProperty("Version", "1.0")]
[TestProperty("Modulo", "Usuarios")]
public void CrearUsuario_DatosValidos_UsuarioCreado()
{
}
```

Casos de uso habituales

Identificar al autor	[TestProperty("Autor", "Pedro")]
Asociar requisitos	[TestProperty("Requisito", "RF-12")]
Asociar casos de uso	[TestProperty("CasoUso", "CU-05")]
Identificar componentes	[TestProperty("Modulo", "Autenticacion")]
Indicar versiones	[TestProperty("Version", "2.1")]

Atributos de control de ejecución

Mientras que las categorías permiten organizar pruebas, los atributos de control de ejecución modifican el comportamiento del framework durante la ejecución: ***[Ignore]*** y ***[Timeout]***.

Atributo Ignore

Permite excluir temporalmente una prueba sin documentar el motivo …

```
[TestMethod]
[Ignore]
public void FuncionalidadPendiente()
{
}
```

… o documentando el motivo

```
[TestMethod]
[Ignore("Pendiente de implementación")]
public void FuncionalidadPendiente()
{
}
```

¿Cuándo utilizar Ignore?

- Funcionalidad aún no desarrollada
- Se trate de un defecto conocido
- Exista una dependencia externa no disponible

¿Cuándo NO utilizar Ignore?

Es frecuente encontrar suites de pruebas con numerosos ***[Ignore]*** utilizados para ocultar errores. Esto es una mala práctica porque los defectos dejan de detectarse automáticamente. No te engañes o peor no engañes.

Atributo [Timeout]

Establece un tiempo máximo de ejecución expresado en milisegundos. Si se supera el límite, la prueba falla automáticamente.

```
[TestMethod]
[Timeout(2000)]
public void Operacion_Compleja_FinalizaEnMenosDeDosSegundos()
{
}
```

Utilidad

- Detectar bloqueos.
- Detectar bucles infinitos.
- Limitar el tiempo de respuesta esperado.

Excepciones

Atributo [ExpectedException(typeof(Exception))]

Indica que una prueba debe finalizar lanzando una excepción determinada y, por lo tanto, la prueba será correcta únicamente si se produce la excepción especificada.

```
[TestMethod]
[ExpectedException(typeof(ArgumentNullException))]
public void Constructor_NombreNulo_LanzaExcepcion()
{
    new Usuario(null);
}
```

Limitaciones

- Cuando hay más de una instrucción puede incurrir en inconsistencia al no conocer cuál de ellas produce la excepción especificada.

Por este motivo actualmente suele preferirse la "Aserción":

```
[TestMethod]
public void Constructor_NombreNulo_LanzaExcepcion()
{
    Assert.ThrowsException<ArgumentNullException>(
        () => new Usuario(null));
}
```

Principales aserciones de MSTest

Las aserciones (assertions) son los mecanismos utilizados para verificar que el comportamiento observado coincide con el comportamiento esperado. Una prueba únicamente puede determinar si ha tenido éxito o ha fallado cuando incluye una o más aserciones. MSTest proporciona principalmente la clase Assert para este propósito.

Assert.AreEqual

Comprueba que dos valores son iguales.

```
[TestMethod]
public void Sumar_DosMasTres_DevuelveCinco()
{
    int resultado = 2 + 3;

    Assert.AreEqual(5, resultado);
}
```

Uso típico:

Se usa para comparar cualquier objeto que implementan correctamente Equals().

- Valores numéricos.
- Cadenas de texto.
- Enumeraciones.

Assert.AreNotEqual

Comprueba que dos valores son distintos.

```
[TestMethod]
public void Usuario_NombreNoDebeSerAdmin()
{
    string nombre = "Pedro";

    Assert.AreNotEqual("Admin", nombre);
}
``
```

Assert.IsTrue

Verifica que una expresión booleana sea verdadera.

```
[TestMethod]
public void Usuario_EsMayorDeEdad()
{
    int edad = 20;

    Assert.IsTrue(edad >= 18);
}
```

Otras aserciones

Otras aserciones básicas

Aserción	Descripción	Ejemplo
Assert.AreEqual()	Comprueba que dos valores son iguales.	Assert.AreEqual(5, resultado);
Assert.AreNotEqual()	Comprueba que dos valores son distintos.	Assert.AreNotEqual(0, resultado);
Assert.IsTrue()	Verifica que una condición sea verdadera.	Assert.IsTrue(edad >= 18);
Assert.IsFalse()	Verifica que una condición sea falsa.	Assert.IsFalse(usuario.Bloqueado);
Assert.IsNull()	Comprueba que una referencia es nula.	Assert.IsNull(usuario);
Assert.IsNotNull()	Comprueba que una referencia no es nula.	Assert.IsNotNull(usuario);
Assert.AreSame()	Comprueba que dos referencias apuntan al mismo objeto.	Assert.AreSame(u1, u2);
Assert.AreNotSame()	Comprueba que dos referencias apuntan a objetos distintos.	Assert.AreNotSame(u1, u2);
Assert.IsInstanceOfType<T>()	Comprueba que un objeto es de un tipo determinado.	Assert.IsInstanceOfType<Usuario>(obj);
Assert.IsNotInstanceOfType<T>()	Comprueba que un objeto no es de un tipo determinado.	Assert.IsNotInstanceOfType<Administrador>(obj);
Assert.ThrowsException<T>()	Verifica que se lanza una excepción específica.	Assert.ThrowsException<ArgumentException>(() => Metodo());
Assert.Fail()	Fuerza el fallo de la prueba.	Assert.Fail("No debería alcanzarse este punto");
Assert.Inconclusive()	Indica que la prueba no puede determinar un resultado concluyente.	Assert.Inconclusive("Servicio no disponible");

Aserción	Descripción	Ejemplo
Assert.That()	Evalúa una expresión booleana (MSTest 4).	Assert.That(total > 0);

Aserciones para cadenas

Aserción	Descripción	Ejemplo
StringAssert.Contains()	Comprueba que una cadena contiene otra.	StringAssert.Contains(texto, "UBU");
StringAssert.StartsWith()	Comprueba el prefijo de una cadena.	StringAssert.StartsWith(texto, "Pedro");
StringAssert.EndsWith()	Comprueba el sufijo de una cadena.	StringAssert.EndsWith(fichero, ".pdf");

Aserciones para colecciones

Aserción	Descripción	Ejemplo
CollectionAssert.Contains()	Comprueba que una colección contiene un elemento.	CollectionAssert.Contains(lista, usuario);
CollectionAssert.DoesNotContain()	Comprueba que un elemento no pertenece a la colección.	CollectionAssert.DoesNotContain(lista, usuario);
CollectionAssert.AreEqual()	Comprueba que dos colecciones son idénticas y están en el mismo orden.	CollectionAssert.AreEqual(esperada, real);
CollectionAssert.AreEquivalent()	Comprueba que dos colecciones contienen los mismos elementos, sin importar el orden.	CollectionAssert.AreEquivalent(esperada, real);
CollectionAssert.AllItemsAreNotNull()	Comprueba que ningún elemento es nulo.	CollectionAssert.AllItemsAreNotNull(lista);

Enlaces de interés sobre aserciones

Microsoft mantiene la documentación actualizada de todas las aserciones de MSTest en:

1. Aserciones de MSTest (Microsoft Learn)
2. MSTest Assertions (versión en inglés)

Estas páginas incluyen la lista completa de métodos de Assert, ejemplos de uso y novedades introducidas en las versiones más recientes de MSTest.

Microsoft Code Coverage

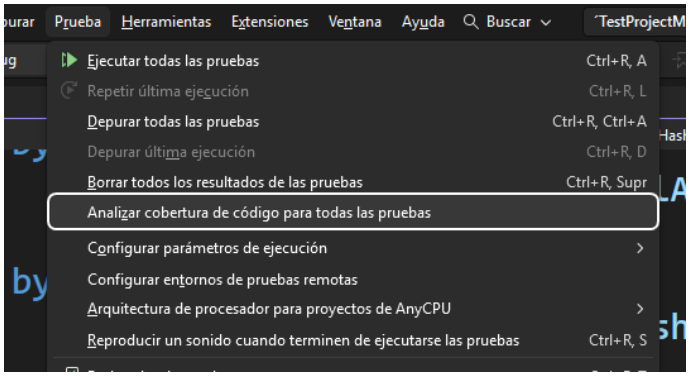
Microsoft Code Coverage es la herramienta de análisis de cobertura integrada en el ecosistema moderno de pruebas de Microsoft. Su función consiste en medir qué partes del código fuente han sido ejecutadas durante la ejecución de las pruebas automatizadas. Gracias a esta información es posible identificar métodos, ramas o líneas de código que todavía no están siendo verificadas por las pruebas, ayudando a mejorar la calidad y la fiabilidad del software.

Microsoft Code Coverage no crea ni ejecuta pruebas. Trabaja juntamente con Microsoft Testing Platform, observando qué código se ejecuta mientras las pruebas definidas con MSTest se están ejecutando.

Ejecutar las pruebas con análisis de cobertura

Desde Visual Studio

Desde Visual Estudio: “Prueba → Analizar cobertura de código para todas las pruebas”



Desde la línea de comandos:

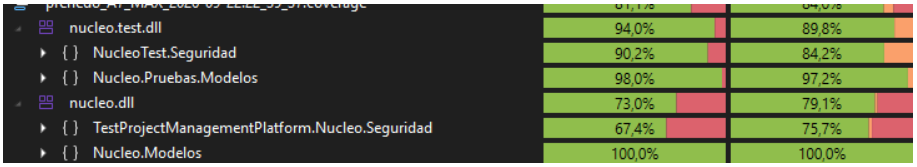
```
dotnet test --coverage
```

Una vez finalizada la ejecución, Visual Studio mostrará un informe indicando qué porcentaje del código ha sido cubierto por las pruebas.

Analizar los resultados

Los informes de cobertura suelen mostrar:

- Número de líneas ejecutadas.
- Número de líneas no ejecutadas.
- Porcentaje de cobertura.
- Métodos sin pruebas.
- Clases parcialmente cubiertas.



Localizar métodos que no han sido ejecutados durante las pruebas

Una de las tareas más habituales consiste en localizar métodos que no han sido ejecutados por ninguna prueba.

Si ningún caso de prueba invoca este método, aparecerá en el informe como código no cubierto.

Mejorar la cobertura

Tras identificar las zonas sin cubrir, se crean nuevas pruebas que ejerciten:

- Casos normales.
- Casos límite.
- Condiciones de error.
- Excepciones.

Interpretación de los resultados

Una cobertura alta no garantiza la ausencia de defectos, simplemente significa que todo el código ha sido ejecutado al menos una vez.

Esto no significa que:

1. Todas las pruebas sean correctas.
2. Se hayan comprobado todos los resultados posibles.
3. No existan errores lógicos.

Por este motivo, la cobertura debe utilizarse como un indicador de ayuda y no como una medida absoluta de calidad.

Excluir código del análisis

Microsoft Code Coverage permite tanto excluir código del análisis como visualizar qué partes están completamente cubiertas, no cubiertas o parcialmente cubiertas.

En determinadas situaciones no resulta útil medir la cobertura de ciertos elementos, por ejemplo:

- Código generado automáticamente.
- Clases DTO o entidades simples.
- Código de terceros.
- Clases temporales de prueba.

Puede utilizarse el atributo:

Atributo [ExcludeFromCodeCoverage]

Se puede incluir el atributo ***[ExcludeFromCodeCoverage]*** delante del atributo de clase o aplicarse sobre métodos concretos,

```
[ExcludeFromCodeCoverage]
public static string ObtenerVersion()
{
    return "1.0";
}
```

Cómo se visualiza la cobertura

Visual Studio puede mostrar información a nivel de:

- Ensamblado.

- Espacio de nombres.
- Clase.
- Método.
- Línea de código.
- Código completamente cubierto

Se considera cubierto cuando todas las instrucciones han sido ejecutadas durante las pruebas.

Representación de la cobertura

Tras ejecutar el análisis de cobertura, Visual Studio resalta el código fuente utilizando diferentes colores. El color verde indica código completamente cubierto por las pruebas, el rojo identifica código no ejecutado durante las pruebas y el amarillo muestra código parcialmente cubierto, normalmente debido a estructuras de decisión (if, switch, etc.) cuyas ramas no han sido verificadas completamente. Estas indicaciones permiten identificar rápidamente qué partes de la aplicación requieren nuevas pruebas unitarias.

Visual Studio puede resaltar el código cubierto y no cubierto mediante colores configurables. La representación exacta depende de la versión, el tema y la configuración del entorno. Por ello, debe consultarse la leyenda mostrada por la ventana de resultados de cobertura.

Explorador de pruebas

El **Explorador de pruebas** es la herramienta de Visual Studio que permite descubrir, ejecutar, depurar y analizar pruebas de forma gráfica. Actúa como una interfaz sobre Microsoft Testing Platform, mostrando las pruebas definidas mediante MSTest y los resultados obtenidos tras su ejecución.

Apertura del Explorador de pruebas

Puede abrirse desde:

1. Prueba → Explorador de pruebas
2. Ver → Explorador de pruebas

Principales funcionalidades

Descubrir pruebas

Visual Studio detecta automáticamente todas las pruebas que contienen atributos como:

```
[TestClass]
[TestMethod]
```

Las pruebas aparecen agrupadas en el Explorador de pruebas.

Ejecutar pruebas

Permite ejecutar:

- Todas las pruebas.
- Una única prueba.
- Un conjunto de pruebas seleccionado.

- Las pruebas filtradas.

Depurar pruebas

Permite iniciar una prueba en modo depuración.

- Depurar pruebas seleccionadas

Los puntos de ruptura (*breakpoints*) funcionan igual que durante la ejecución normal de una aplicación.

Filtrar pruebas

Las pruebas pueden filtrarse por:

- Nombre.
- Proyecto.
- Estado.
- Categoría (TestCategory).

Consultar resultados

Tras la ejecución, cada prueba mostrará su estado:

Icono	Significado
	Prueba superada
	Prueba fallida
	Prueba ignorada
	Resultado no concluyente

Al seleccionar una prueba fallida se muestran:

- Mensaje de error.
- Excepción producida.
- Traza de ejecución (*stack trace*).